

RailRoute — COA Task 1

Documentation

Processor-Level Modeling of Application (CO1)

File: src/coa_simulator/Task1_ProcessorModeling.asm

1. Register / ALU / Control Unit / Memory Table

Component	Role in this program
Memory (DATA segment)	Holds SEG1_PUNE_NASHIK (210), SEG2_NASHIK_MUMBAI (165), TOTAL_DISTANCE (result) — the RailRoute route data as it sits in RAM before/after processing
Register (AX)	Holds the operand currently being worked on — loaded from memory, then used as the ALU's destination
ALU	Executes the ADD AX, SEG2_NASHIK_MUMBAI instruction — the binary adder circuit that produces $210+165=375$
Control Unit	Sequences the fetch-decode-execute cycle for each instruction: decides “load from memory,” “route to ALU,” “store to memory” in order

2. Instruction Explanation

```
MOV AX, SEG1_PUNE_NASHIK    ; memory -> register (Control Unit
                             routes the fetch)
ADD AX, SEG2_NASHIK_MUMBAI  ; ALU adds register + memory operand
MOV TOTAL_DISTANCE, AX      ; register -> memory (Control Unit
                             routes the store)
```

MOV is a data-movement instruction — no ALU involvement, purely a Control Unit-directed transfer. ADD is the one instruction here that actually uses the ALU: it takes AX and the memory operand as inputs and produces a new AX value. This program does not need comparison or branching — CO1 only asks for one arithmetic operation mapped to processor components; comparison/branching is CO2's requirement, already covered in Task 2.

3. RailRoute-to-COA Mapping

Element	RailRoute meaning
SEG1_PUNE_NASHIK (210), SEG2_NASHIK_MUMBAI (165)	Real segment distances from RouteManager (same values used later in Task 2 Operation 2's

TOTAL_DISTANCE (375)	Route B) The combined Pune→Nashik→Mumbai leg distance — the same intermediate figure the app-level “Shortest Route” logic would need
----------------------	--

Deliberately reuses the same station pair as Task 2 so CO1 and CO2 read as one coherent story in the viva (CO1 = “here’s how a route-distance add looks at the processor level,” CO2 = “here’s the same idea done for real, in 64-bit, plus a comparison”) rather than two unrelated exercises.

4. Test Plan & Status

Test	Expected	Status
AX after MOV AX, SEG1...	210	Not yet run — needs your CPU simulator
AX after ADD	375	Not yet run
TOTAL_DISTANCE in memory	375	Not yet run

I have no access to your CPU simulator, so none of this has been executed or validated by me. Please step through it in your simulator and tell me the actual register/memory values you observe — I’ll only mark this validated once you report real results.

5. Viva Questions & Answers

- What does CO1 ask for that CO2 doesn’t?** CO1 is about *mapping* an operation to processor components (registers/ALU/control unit/memory) conceptually; CO2 is about a *real, complete* 64-bit implementation with comparison and branching.
- Why 16-bit here and 64-bit in Task 2?** CO1’s requirement is architectural understanding, not width; a small simulator-friendly width keeps the demonstration simple. CO2 explicitly mandates 64-bit, which is a native x86-64/NASM concern, not something typical teaching simulators model.
- Which instruction actually uses the ALU?** Only ADD — the two MOVs are pure data transfers directed by the Control Unit, not arithmetic.
- Why the same route pair as Task 2?** To show CO1 and CO2 as one continuous story about the same real RailRoute computation, rather than two disconnected exercises.
- What’s the memory→register→ALU→register→memory path here?** SEG1 (memory) → AX (register) → ADD with SEG2 (ALU + memory operand) → AX (register, updated) → TOTAL_DISTANCE (memory).

6. AI-Use Record (fill in what you actually did)

Field	Content
AI tool used	Claude (Anthropic) Drafting the CO1 processor-level

Purpose	model and mapping it to RailRoute data
Prompt(s) used	<i>(paste your actual prompts)</i>
Output summary	Task1_ProcessorModeling.asm — 16-bit ADD demonstrating register/ALU/control-unit/memory mapping
Problems/limitations found	Exact simulator syntax (“YASMIN”) could not be verified — standard 8086-style syntax used as a best-effort default; you must confirm/adjust it against your actual simulator before submission
Corrections YOU actually made	<i>(fill in only if you changed anything)</i>
Tests YOU performed	<i>(fill in once you’ve run it in your simulator)</i>
Final validation	<i>(pending your own execution)</i>
